OpenAPI Documentation in Terms of Architectural Specs

- Swagger/OpenAPI is **not a full architecture documentation system** (it won't replace ADRs, C4 diagrams, DB schema docs, deployment runbooks).
- It *is* excellent at documenting the **API surface as an architectural contract** — which is often the most critical part for teams to align on.
- Swagger (more precisely: **Swagger UI + the Swagger tooling ecosystem**) can absolutely help you document *parts* of a project's architecture — **especially the "integration architecture"**: module boundaries exposed over HTTP, request/response contracts, auth patterns, error envelopes, versioning, and even async callbacks/webhooks.

☑ What Swagger/OpenAPI is great for (architecture-as-contract)

OpenAPI describes the API surface in a standard format and can be consumed by documentation tools, code generators, and test tools.
Use it to document:

- **Module / bounded-context boundaries** via tags (e.g., Users, Orders, Payments). OpenAPI supports a top-level tags section used by tools to organize operations.
- **Authentication & authorization architecture** via securitySchemes + global/per-operation security.
- **Error model + envelopes** as shared components.schemas referenced everywhere.
- **Environment topology** (dev/stage/prod) via servers.
- **Cross-cutting conventions** like pagination, id formats, correlation IDs, idempotency keys (documented as headers/parameters + reusable components).
- **Async interactions** using callbacks/webhooks (OpenAPI 3.1 includes webhooks at the root level, and also defines callbacks).

❌ What it's not designed for

- Internal class diagrams, DB physical schema, deployment diagrams, threading models, etc. (you *link* those from OpenAPI instead of embedding them).

The "bridge" concept: OpenAPI spec + Swagger UI + external architecture docs

The pattern that works best in real projects:
Use OpenAPI as the canonical contract for external behavior, and link to deeper architecture docs (ADRs, C4, sequence diagrams) from the spec.

OpenAPI has an externalDocs field at the root that explicitly supports **linking to external documentation**.
Swagger UI renders OpenAPI specs into interactive documentation automatically.
So your architecture docs become "two-layered":

1. **Interactive contract docs** (Swagger UI)
2. **Conceptual / structural docs** (ADRs, C4 diagrams, runbooks), linked from the contract

How to encode architecture information into OpenAPI (practical techniques)

A) Use Tags to mirror module boundaries (modular monolith)
In a modular monolith, tags map cleanly to modules/bounded contexts: Users, Orders, Auth, etc.
OpenAPI supports tags and tools use them for grouping.
What to document architecturally with tags:

- Ownership (team/module)
- Responsibilities
- Stability (stable/beta/internal)
- Allowed dependencies (who is allowed to call what)

B) Use externalDocs to link to architecture diagrams/ADRs
OpenAPI supports externalDocs for linking external documentation at the API level.
Example: link to /docs/architecture/ where you keep:

- C4 Context/Container diagrams
- ADRs (auth decisions, caching strategy, eventing choices)
- Sequence diagrams for key flows

C) Use Specification Extensions (x-...) for architecture metadata
OpenAPI supports **extensions** (a.k.a. vendor/spec extensions) as fields prefixed with x-, usable at multiple levels (root/info/paths/operations/tags/security schemes, etc.).
This is the "secret weapon" for architectural specs because you can add metadata that OpenAPI doesn't model natively, like:

- x-owner-team
- x-bounded-context
- x-rate-limit-tier
- x-slo-latency-p95
- x-data-classification
- x-internal: true

Most tooling ignores unknown x- fields safely, while some tools can render them with customization.
D) Document cross-cutting concerns explicitly
OpenAPI supports:

- Reusable schemas in components
- Global security definitions
- Standard response objects

- Server environments
  All of these help you express architectural conventions once, and reuse everywhere.

How this looks in a real NestJS modular monolith

NestJS commonly generates an OpenAPI doc via @nestjs/swagger, then serves Swagger UI from it.
Approach

- Each module's controller uses @ApiTags('ModuleName') to reflect architecture boundaries.
- At bootstrap, create the OpenAPI doc and:
  - Set global info/servers/security (cross-cutting architecture).
  - Optionally **post-process** the OpenAPI JSON to add externalDocs and x-... fields.

NestJS snippet: generate OpenAPI + inject architecture metadata

```
5  async function bootstrap() {
6    const app = await NestFactory.create(AppModule);
7
8    const config = new DocumentBuilder()
9      .setTitle('Service API')
10     .setDescription('Contract + architecture entry point')
11     .setVersion('1.0.0')
12     .addBearerAuth({ type: 'http', scheme: 'bearer', bearerFormat: 'JWT' }, 'jwt')
13     .build();
14
15   const doc = SwaggerModule.createDocument(app, config, { deepScanRoutes: true }); //
     generates OpenAPI doc [5](https://www.codestudy.net/blog/generate-swagger-documentation-as-
     json-yaml-in-nestjs/)
16
17   // Inject architecture-level links & metadata (OpenAPI supports externalDocs + x-
     extensions) [2](https://spec.openapis.org/oas/v3.1.0.html)[4]
     (https://swagger.io/docs/specification/v3_0/openapi-extensions/)
18   doc.externalDocs = {
19     description: 'Architecture docs (C4 + ADRs + runbooks)',
20     url: 'https://your-repo/wiki/architecture',
21   };
22
23   doc['x-architecture'] = {
24     style: 'modular-monolith',
25     boundedContexts: ['Users', 'Orders', 'Payments'],
26     conventions: {
27       errorEnvelope: 'ErrorEnvelope',
28       auth: 'JWT Bearer',
29     },
30   };
31
32   SwaggerModule.setup('docs', app, doc); // Swagger UI renders from OpenAPI [1]
     (https://swagger.io/tools/swagger-ui/)[5](https://www.codestudy.net/blog/generate-swagger-
     documentation-as-json-yaml-in-nestjs/)
33
34   await app.listen(3000);
35 }
36 bootstrap();
```

Flow diagram: turning architecture specs into “living docs”

```
            +----------------------+
            |  ADRs / C4 Diagrams  |
            |  Runbooks / RFCs     |
            +----------+----------+
                       ^
                       |  externalDocs links (OpenAPI supports this)
                       |  + x- extensions for metadata
                       |
+----------------------+----------------------+
|             OpenAPI Spec (JSON/YAML)        |
|  - tags = module boundaries                 |
|  - components = shared contracts            |
|  - securitySchemes = auth architecture      |
|  - servers = env topology                   |
+----------------------+----------------------+
                       |
                       | rendered by Swagger UI from OpenAPI
                       v
            +----------------------+
            |      Swagger UI      |
            | interactive contract |
            | "Try it out" + models |
            +----------------------+
```

Make it “official architecture”: versioning & publishing

Architecture specs matter only if they are:

- versioned,
- reviewed,
- and published somewhere reliable.

SwaggerHub supports “publishing” API definitions as stable, read-only versions and encourages creating new versions for changes.

So a practical workflow is:

1. Generate openapi.json in CI (from NestJS).
2. Publish it (docs site / SwaggerHub / internal portal).
3. Treat spec changes like architecture changes (review diffs in PR).

Publishing as “stable contract” is exactly the kind of governance that turns API docs into architecture documentation.

Bottom line

Swagger isn’t a general architecture doc tool — **but it’s an excellent way to document architecture at the boundaries** (contracts, auth, error models, cross-cutting conventions).
The strongest pattern is:

- **OpenAPI = the contract**
- **Swagger UI = the interactive presentation**
- **externalDocs + x-extensions = the glue to your deeper architecture docs**